

FUNDAMENTAL OF COMPUTER PROGRAMMING

Definition of terms : Computer programming

- ▶ A programming language defines a set of instructions that are compiled together to perform a specific task by the CPU (Central Processing Unit).
- ▶ **A computer program is a set of instructions used to operate a computer to produce a specific result.**
- ▶ Each programming language contains a unique set of keywords and syntax, which are used to create a set of instructions. Thousands of programming languages have been developed till now, but each language has its specific purpose.
- ▶ These languages vary in the level of abstraction they provide from the hardware. Some programming languages provide less or no abstraction while some provide higher abstraction.
- ▶ Based on the levels of abstraction, they can be classified into two categories: low-level and high-level language.

1. Low-level language

- ▶ The low-level language is a programming language that provides no abstraction from the hardware, and it is represented in 0 or 1 forms, which are the machine instructions.
- ▶ The languages that come under this category are the
 - ▶ Machine level language
 - ▶ Assembly language.

a. Machine languages

- ▶ The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1.
- ▶ As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so the instructions given to the computer can be only in binary codes.
- ▶ Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions.
- ▶ It is error-prone as it is not easy to understand, and its maintenance is also very high.
- ▶ A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.

Machine languages (cont.)

- ▶ A machine language instruction consists of two parts: an instruction part and an address part.
- ▶ The instruction part (opcode) tells the computer the operation to be performed.
- ▶ The address part specifies the memory address of the data to be used in the instruction.

b. Assembly languages

- ▶ **Assembly languages perform the same tasks as machine languages, but use symbolic names for opcodes and operands instead of 1s and 0s.**
- ▶ The assembly language contains some human-readable commands such as mov, add, sub, etc.
- ▶ The problems which we were facing in machine-level language are reduced to some extent by using an extended form of machine-level language known as assembly language.
- ▶ Since assembly language instructions are written in English words like mov, add, sub, so it is easier to write and understand. For example ADD OVERPAY, STORE GROSSPAY
- ▶ An assembly language program must be translated into a machine language program before it can be executed on a computer.

Assembler

- ▶ An assembler is a program that converts assembly language into machine code.
- ▶ It takes the basic commands and operations from assembly code and converts them into binary code that can be recognized by a specific type of processor



Difference between machine language and assembly language

Machine-level language	Assembly language
The machine-level language comes at the lowest level in the hierarchy, so it has zero abstraction level from the hardware.	The assembly language comes above the machine language, which means that it has less abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the computer.	In assembly language, the assembler is used to convert assembly code into machine code.

Advantages of low level languages

- ▶ Programs developed using low level languages are fast and memory efficient.
- ▶ Programmers can utilize processor and memory in better way using a low level language.
- ▶ There is no need of any compiler or interpreters to translate the source to machine code. Thus, cuts the compilation and interpretation time.
- ▶ Low level languages provide direct manipulation of computer registers and storage.
- ▶ It can directly communicate with hardware devices.

Disadvantages of low level languages

- ▶ Programs developed using low level languages are machine dependent and are not portable.
- ▶ It is difficult to develop, debug and maintain.
- ▶ Low level programs are more error prone.
- ▶ Low level programming usually results in poor programming productivity.
- ▶ Programmer must have additional knowledge of the computer architecture of particular machine, for programming in low level language.

2. High level languages

- ▶ Computer programs are usually written on high level languages. A high level language is one that can be understood by humans for example English.
- ▶ However, computers cannot understand high level languages as we humans do. They can only understand the programs that are developed in binary systems known as a machine code.
- ▶ A computer program written in high level language is known as a **source code**.
- ▶ These source codes must be converted into machine language, hence the introduction of Compilers and interpreters.
- ▶ Compilers and interpreters are programs that help convert the high level language (Source Code) into machine codes that can be understood by the computers.
- ▶ However there exists differences between the two programs as depicted below:

Differences between Compilers and interpreters

Interpreter translates just one statement of the program at a time into machine code.

An interpreter takes very less time to analyze the source code. However, the overall time to execute the process is much slower.

An interpreter does not generate an intermediary code. Hence, an interpreter is highly efficient in terms of its memory.

Keeps translating the program continuously till the first error is confronted. If any error is spotted, it stops working and hence debugging becomes easy.

Interpreters are used by programming languages like Ruby and Python for example.

Compiler scans the entire program and translates the whole of it into machine code at once.

A compiler takes a lot of time to analyze the source code. However, the overall time taken to execute the process is much faster.

A compiler always generates an intermediary object code. It will need further linking. Hence more memory is needed.

A compiler generates the error message only after it scans the complete program and hence debugging is relatively harder while working with a compiler.

Compilers are used by programming languages like C and C++ for example.

High level languages

- ❖ The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer. The high-level languages are considered as high-level because they are closer to human languages than machine-level languages
- ❖ They allow the programmer to focus on what needs to be done, rather than on how the computer actually works.
- ❖ For example, in many high-level languages, to place a message on the screen, a programmer would use the statement `'print'`.
- ❖ The programmer might not know how the computer actually generates the message. They just need to know how to use the `'print'` statement.

High level languages

- ▶ Many types of high-level language exist and are in common use today, including:
 - ❖ Python
 - ❖ Java
 - ❖ C++
 - ❖ C#
 - ❖ Visual Basic
 - ❖ JavaScript
- ▶ **Source code** is the term given to a set of instructions that are written in human readable programming language. Source code must be translated into machine code before a computer can understand and execute it.

Advantages of High level languages

- ▶ High level languages are programmer friendly.
- ▶ They are easy to write, debug and maintain.
- ▶ It provide higher level of abstraction from machine languages.
- ▶ It is machine independent language.
- ▶ Easy to learn.
- ▶ Less error prone, easy to find and debug errors.
- ▶ High level programming results in better programming productivity

Programming Paradigms

- ❖ The term **programming paradigm** is used to specify an overall approach to writing program code.
- ❖ There are basically four programming paradigms in programming languages:
 - 1) Procedural programming languages
 - 2) Functional programming languages
 - 3) Logic programming languages
 - 4) Object-oriented programming languages

Others include

- 5) Event-Driven programming languages
- 6) Concurrent / parallel programming languages
- 7) Special purpose programming languages

1. Procedural programming paradigm

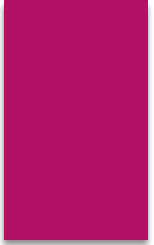
- ❖ The **procedural programming paradigm** is where program code is divided up into **procedures**, discrete blocks of code that carry out a single task.
- ❖ Procedures, also known as routines, subroutines, or functions, simply contain a series of computational steps to be carried out in the order specified by the programmer.
- ❖ The different terms used for procedures can be confusing. In some programming languages, the term **function** is used for a subroutine that returns one or more values, and the term **procedure** is used for a piece of code that does something, but does not return any values.
- ❖ Example of programming languages under this paradigm include C, C++, Pascal, Python etc

2. Functional programming languages

- ❖ It is also known as applicative languages.
- ❖ Functional programming is a form of declarative programming (mathematical function).
- ❖ Programming consists of building the function that computes the answer.
- ❖ Example of Functional language is: Lisp, Python, Haskell

3. Logic programming languages:

- ❖ This type of languages concentrate on what is the expected outcome for the program instead of how the outcome is achieved. Logical programming is something like math .Logic program statements express facts and rules about problems. To understand the rules, lets take an example like, "A is true if B and C is true".
- ❖ And to understand facts we can say that "A is true".
- ❖ Example of Logic language is: Prolog.



4) Object-oriented programming languages:

Object oriented programming language based on object instead of just functions and procedures.

It involves concepts of oops programming languages.

For example: C++, Java.

5) Event Driven programming languages:

These languages execute various operations based on user activities like mouse click and other events.

For example: Visual Basic, Java.

- Many programming languages support multiple paradigms. Python, which is a popular general-purpose programming language, allows you to code using procedural, object-oriented, or functional techniques.

Flow charts

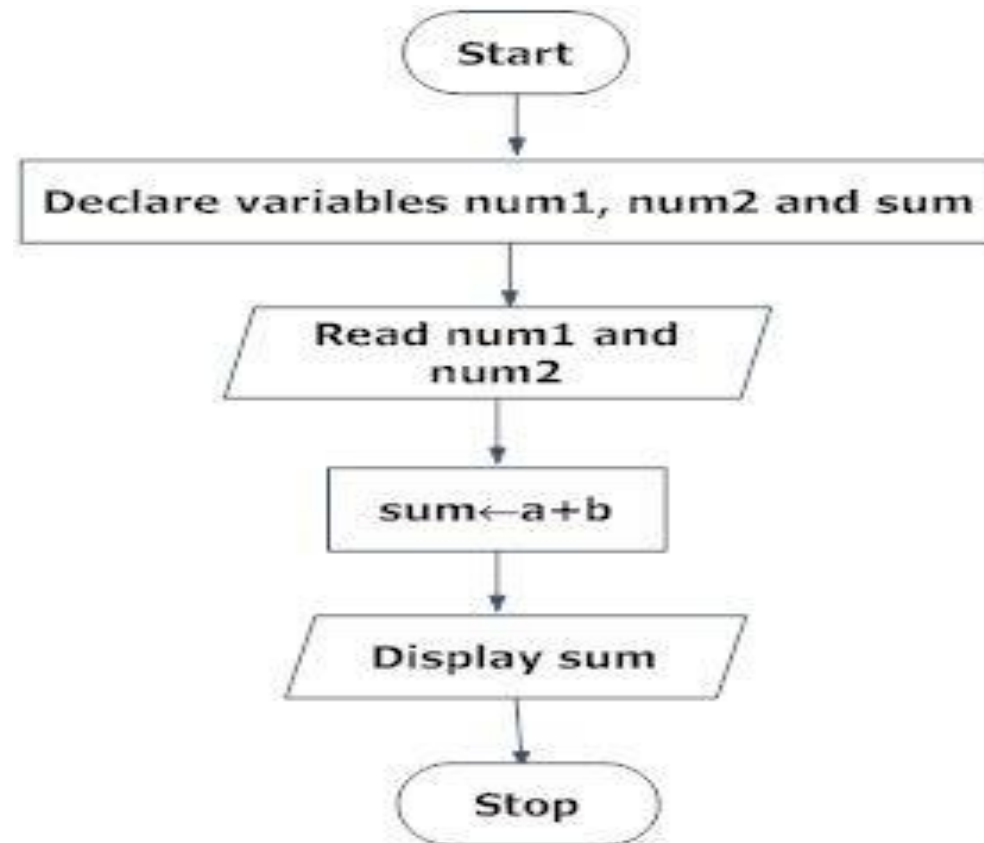
- ▶ A flowchart is a graphical representations of steps.
- ▶ It shows steps in sequential order and is widely used in presenting the flow of algorithms, workflow or processes.
- ▶ It originated from computer science as a tool for representing algorithms and programming logic but had extended to use in all other kinds of processes.
- ▶ Nowadays, flowcharts play an extremely important role in displaying information and assisting reasoning.
- ▶ They help us visualize complex processes, or make explicit the structure of problems and tasks. A flowchart can also be used to define a process or project to be implemented

Flowchart Symbols

Symbol	Purpose	Description
	Flow line	Used to indicate the flow of logic by connecting symbols.
	<u>Terminal</u> (Stop/Start)	Used to represent start and end of flowchart.
	<u>Input/Output</u>	Used for input and output operation.
	Processing	Used for arithmetic operations and data-manipulations.
	Decision	Used to represent the operation in which there are two alternatives, true and false.
	On-page Connector	Used to join different flowline
	Off-page Connector	Used to connect flowchart portion on different page.
	Predefined Process/Function	Used to represent a group of statements performing one processing task.

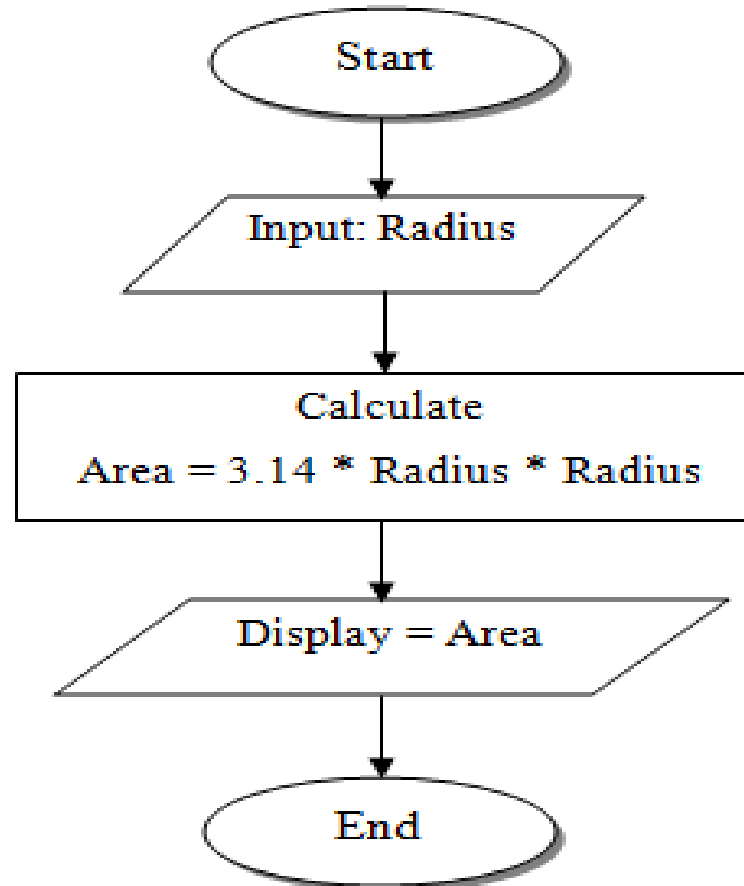
Example of flow charts

Draw a flowchart to
add two numbers



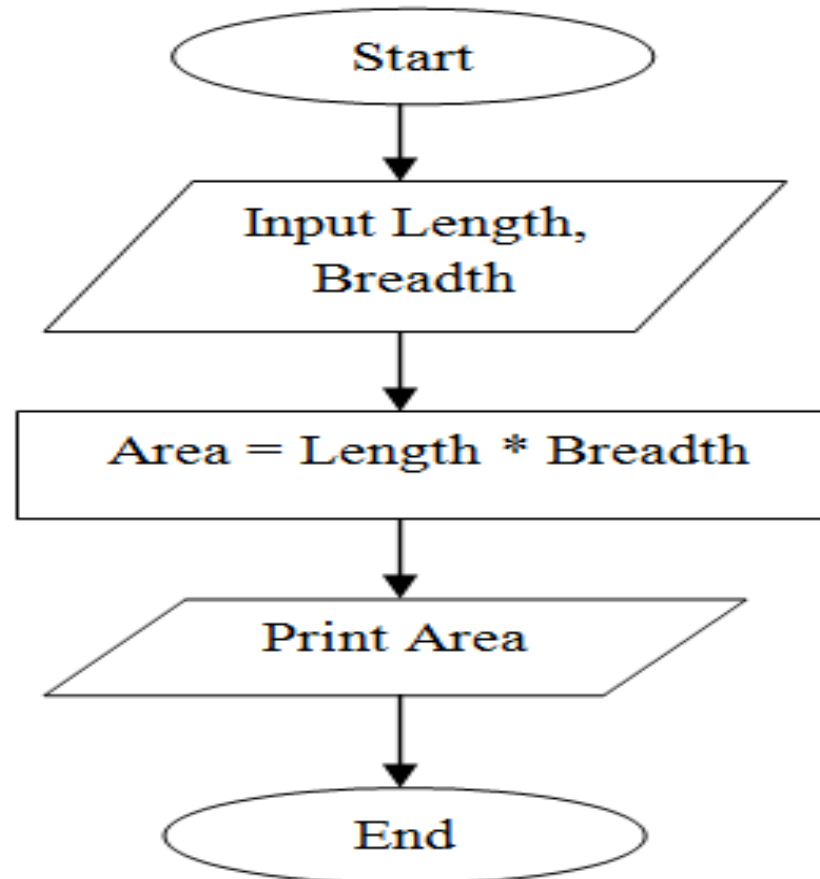
Example of flow charts

Draw a flowchart to find area of a circle



Example of flow charts

Draw a flowchart to find area of a Rectangle



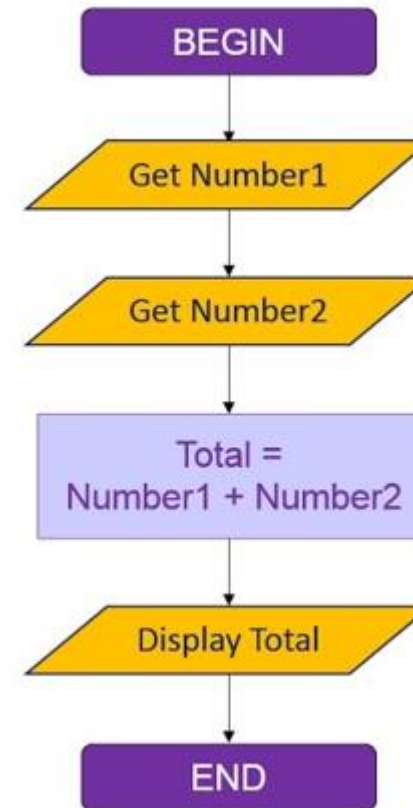
CONTROL STRUCTURES

- ❖ Control Structures can be considered as the building blocks of computer programs.
- ❖ They are commands that enable a program to “take decisions”, following one path or another
- ❖ There are three control structures
 - ❖ Sequence
 - ❖ Iteration (loop)
 - ❖ Selection (conditionals)

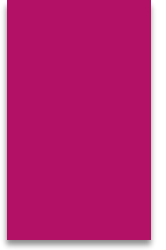
CONTROL STRUCTURES

Sequence control structure refers to the line by line execution by which statements are executed sequentially, in the same order in which they appear in the program. They might, for example, carry out a series of read and write operations, arithmetic operations or assignments to variables.

The sequence control structure is the simplest of the three fundamental control structures.



CONTROL STRUCTURES



Selection/conditional control structure

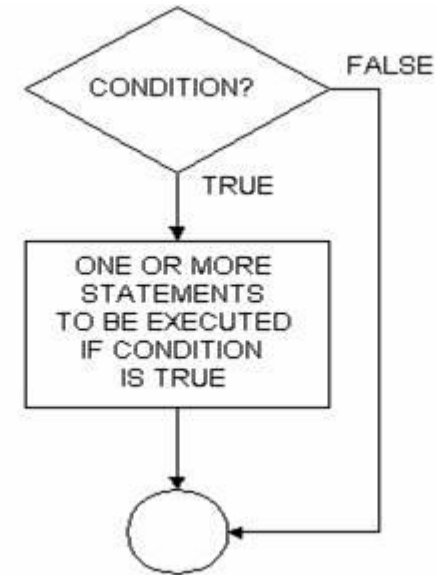
“Conditionals” are at the very core of programming. The idea behind them is that they allow you to control the flow of the code that is executed based on different conditions in the program (e.g. an input taken from the user, or the internal state of the machine the program is running on).

There are three selection structures

- ❖ If ... then
- ❖ If ... then ...else
- ❖ Nested if

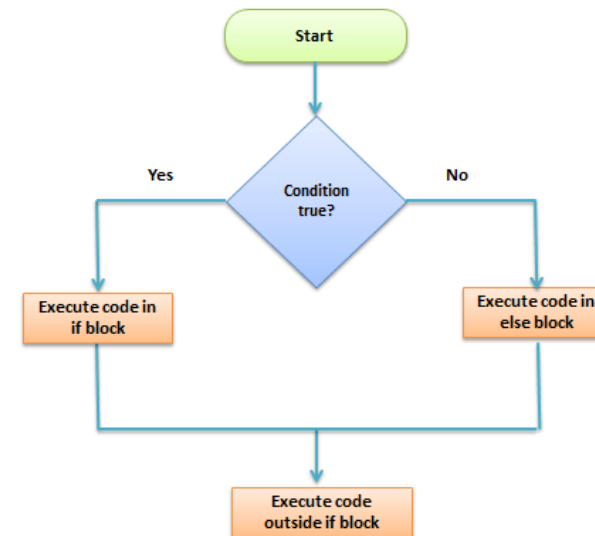
IF selection structure

- ❖ “If statements” execute one or more statements when a condition is met. If the testing of that condition is TRUE, the statement gets executed. But if it’s FALSE (the condition is not met), then nothing happens.



IF .. Else selection structure

- ❖ An **else** statement can be combined with an **if** statement.
- ❖ An **else** statement contains the block of code that executes if the conditional expression in the if statement resolves a FALSE value



IF ... ELSEIF ... ELSE

A selection structure used when testing more than one condition on a variable/input

